

```
import os

print(os.listdir("/content"))

['.config', 'outputs', 'day.csv', 'sample_data']
```

```
import pandas as pd

df = pd.read_csv("/content/day.csv")

df.head()
```

|   | instant | dteday     | season | yr | mnth | holiday | weekday | workingday | weathersit | temp     | atemp    | hum      | windspeed | casual | re |
|---|---------|------------|--------|----|------|---------|---------|------------|------------|----------|----------|----------|-----------|--------|----|
| 0 | 1       | 2011-01-01 | 1      | 0  | 1    | 0       | 6       | 0          | 2          | 0.344167 | 0.363625 | 0.805833 | 0.160446  | 331    |    |
| 1 | 2       | 2011-01-02 | 1      | 0  | 1    | 0       | 0       | 0          | 2          | 0.363478 | 0.353739 | 0.696087 | 0.248539  | 131    |    |
| 2 | 3       | 2011-01-03 | 1      | 0  | 1    | 0       | 1       | 1          | 1          | 0.196364 | 0.189405 | 0.437273 | 0.248309  | 120    |    |

```
print(df.columns.tolist())

['instant', 'dteday', 'season', 'yr', 'mnth', 'holiday', 'weekday', 'workingday', 'weathersit', 'temp', 'atemp', 'hum', 'windspe
```

```
import pandas as pd

df = pd.read_csv("/content/day.csv")
```

```
df.head()
```

|   | instant | dteday     | season | yr | mnth | holiday | weekday | workingday | weathersit | temp     | atemp    | hum      | windspeed | casual | re |
|---|---------|------------|--------|----|------|---------|---------|------------|------------|----------|----------|----------|-----------|--------|----|
| 0 | 1       | 2011-01-01 | 1      | 0  | 1    | 0       | 6       | 0          | 2          | 0.344167 | 0.363625 | 0.805833 | 0.160446  | 331    |    |
| 1 | 2       | 2011-01-02 | 1      | 0  | 1    | 0       | 0       | 0          | 2          | 0.363478 | 0.353739 | 0.696087 | 0.248539  | 131    |    |
| 2 | 3       | 2011-01-03 | 1      | 0  | 1    | 0       | 1       | 1          | 1          | 0.196364 | 0.189405 | 0.437273 | 0.248309  | 120    |    |

```
print(df.columns.tolist())

['instant', 'dteday', 'season', 'yr', 'mnth', 'holiday', 'weekday', 'workingday', 'weathersit', 'temp', 'atemp', 'hum', 'windspe
```

```
print("Rows and columns:", df.shape)

Rows and columns: (731, 16)
```

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 731 entries, 0 to 730
Data columns (total 16 columns):
#   Column      Non-Null Count  Dtype
---  -
0   instant     731 non-null   int64
1   dteday      731 non-null   object
2   season      731 non-null   int64
3   yr          731 non-null   int64
4   mnth        731 non-null   int64
5   holiday     731 non-null   int64
6   weekday     731 non-null   int64
7   workingday  731 non-null   int64
8   weathersit   731 non-null   int64
9   temp        731 non-null   float64
10  atemp       731 non-null   float64
11  hum         731 non-null   float64
12  windspeed   731 non-null   float64
```

```
13  casual      731 non-null    int64
14  registered  731 non-null    int64
15  cnt         731 non-null    int64
dtypes: float64(4), int64(11), object(1)
memory usage: 91.5+ KB
```

```
missing = df.isna().sum()
```

```
print(missing)
```

```
instant      0
dteday       0
season       0
yr           0
mnth        0
holiday      0
weekday      0
workingday   0
weathersit    0
temp         0
atemp        0
hum          0
windspeed    0
casual       0
registered   0
cnt          0
dtype: int64
```

```
total_missing = df.isna().sum().sum()
```

```
print("Total missing values:", total_missing)
```

```
Total missing values: 0
```

```
missing_percent = df.isna().mean() * 100
```

```
print(missing_percent)
```

```
instant      0.0
dteday       0.0
season       0.0
yr           0.0
mnth        0.0
holiday      0.0
weekday      0.0
workingday   0.0
weathersit    0.0
temp         0.0
atemp        0.0
hum          0.0
windspeed    0.0
casual       0.0
registered   0.0
cnt          0.0
dtype: float64
```

```
duplicate_rows = df.duplicated().sum()
```

```
print("Duplicate rows:", duplicate_rows)
```

```
Duplicate rows: 0
```

```
duplicate_dates = df["dteday"].duplicated().sum()
```

```
print("Duplicate dates:", duplicate_dates)
```

```
Duplicate dates: 0
```

```
print("Unique dates:", df["dteday"].nunique())
```

```
print("Total rows:", len(df))
```

```
Unique dates: 731
Total rows: 731
```

```
print("season:", sorted(df["season"].unique()))
```

```
print("yr:", sorted(df["yr"].unique()))
```

```
print("mnth:", sorted(df["mnth"].unique()))
```

```
print("holiday:", sorted(df["holiday"].unique()))
print("weekday:", sorted(df["weekday"].unique()))
print("workingday:", sorted(df["workingday"].unique()))
print("weathersit:", sorted(df["weathersit"].unique()))
```

```
season: [np.int64(1), np.int64(2), np.int64(3), np.int64(4)]
yr: [np.int64(0), np.int64(1)]
mnth: [np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8), np.int64(9), np.i
holiday: [np.int64(0), np.int64(1)]
weekday: [np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6)]
workingday: [np.int64(0), np.int64(1)]
weathersit: [np.int64(1), np.int64(2), np.int64(3)]
```

```
print(df[["temp", "atemp", "hum", "windspeed", "cnt"]].describe())
```

|       | temp       | atemp      | hum        | windspeed  | cnt         |
|-------|------------|------------|------------|------------|-------------|
| count | 731.000000 | 731.000000 | 731.000000 | 731.000000 | 731.000000  |
| mean  | 0.495385   | 0.474354   | 0.627894   | 0.190486   | 4504.348837 |
| std   | 0.183051   | 0.162961   | 0.142429   | 0.077498   | 1937.211452 |
| min   | 0.059130   | 0.079070   | 0.000000   | 0.022392   | 22.000000   |
| 25%   | 0.337083   | 0.337842   | 0.520000   | 0.134950   | 3152.000000 |
| 50%   | 0.498333   | 0.486733   | 0.626667   | 0.180975   | 4548.000000 |
| 75%   | 0.655417   | 0.608602   | 0.730209   | 0.233214   | 5956.000000 |
| max   | 0.861667   | 0.840896   | 0.972500   | 0.507463   | 8714.000000 |

```
print("5 days with highest rental counts:")
print(df.nlargest(5, "cnt")[["dteday", "cnt"]])
```

```
5 days with highest rental counts:
   dteday  cnt
623 2012-09-15 8714
637 2012-09-29 8555
630 2012-09-22 8395
447 2012-03-23 8362
504 2012-05-19 8294
```

```
print("5 days with lowest rental counts:")
print(df.nsmallest(5, "cnt")[["dteday", "cnt"]])
```

```
5 days with lowest rental counts:
   dteday  cnt
667 2012-10-29 22
26 2011-01-27 431
725 2012-12-26 441
25 2011-01-26 506
64 2011-03-06 605
```

```
print(df["cnt"].describe())
```

|       |             |
|-------|-------------|
| count | 731.000000  |
| mean  | 4504.348837 |
| std   | 1937.211452 |
| min   | 22.000000   |
| 25%   | 3152.000000 |
| 50%   | 4548.000000 |
| 75%   | 5956.000000 |
| max   | 8714.000000 |

Name: cnt, dtype: float64

```
difference = df["cnt"] - (df["casual"] + df["registered"])

print("Maximum absolute difference:", difference.abs().max())
```

```
Maximum absolute difference: 0
```

```
print("Rows where cnt equals casual + registered:",
      (df["cnt"] == df["casual"] + df["registered"]).sum())

print("Total rows:", len(df))
```

```
Rows where cnt equals casual + registered: 731
Total rows: 731
```

## Research Question

How is daily bike-rental demand associated with observed weather conditions in the UCI Bike Sharing Dataset?

## Stakeholder

Bike-sharing system operator.

## Population

Daily bike-sharing activity represented by the Capital Bikeshare system during 2011–2012.

## Sample

The 731 daily observations contained in `day.csv`.

## Unit of analysis

One calendar day.

## Target

`cnt`, the total number of bike rentals per day.

## Estimand

The association between observed daily weather conditions and total daily bike-rental demand within the observed Capital Bikeshare data from 2011–2012.

## Dataset Provenance

- Dataset: UCI Bike Sharing Dataset
- Source: UCI Machine Learning Repository
- Dataset DOI: 10.24432/C5W894
- License: CC BY 4.0
- File used for analysis: `day.csv`
- Original dataset files downloaded: `day.csv` and `Readme.txt`
- Study period represented in `day.csv`: 2011–2012
- Number of observations used: 731
- Number of variables: 16

The raw `day.csv` file was downloaded from the UCI Machine Learning Repository and used without modifying the original file. The analysis in this notebook uses the daily dataset only.

```
from pathlib import Path
import hashlib

path = Path("/content/day.csv")

sha256 = hashlib.sha256()

with open(path, "rb") as f:
    for chunk in iter(lambda: f.read(1024 * 1024), b''):
        sha256.update(chunk)

print("File:", path.name)
print("Size (bytes):", path.stat().st_size)
print("SHA-256:", sha256.hexdigest())
```

```
File: day.csv
Size (bytes): 57569
SHA-256: a6bcf826782d3c0fbfdcbceead17cd0884185a0dafa8ff10cd48a874ee7ba18be
```

## Frozen Dataset Version

The analysis uses the downloaded `day.csv` file without modification.

- File size: 57,569 bytes
- SHA-256: `a6bcf826782d3c0fbfdcbecad17cd0884185a0dfe8ff10cd48a874ee7ba18be`

This hash identifies the exact raw file used for the analysis and supports reproducibility.

## Data-Generating Process

The observed daily bike-rental count is generated from real-world conditions and user behavior. On each day, weather conditions, season, calendar characteristics, and working-day status may be related to people's decisions to use the bike-sharing system. These decisions result in casual and registered bike rentals, which together form the recorded total rental count (`cnt`).

### DGP chain

Real-world daily conditions

↓

Weather, season, and calendar conditions

↓

People's travel and leisure decisions

↓

Bike-sharing usage

↓

Casual and registered rentals

↓

Recorded total rentals (`cnt`)

↓

Daily dataset (`day.csv`)

## Provenance Lineage

The provenance of the analyzed data can be described as:

Capital Bikeshare system ↓ Original dataset creators/researchers ↓ UCI Bike Sharing Dataset ↓ UCI Machine Learning Repository ↓

Downloaded `day.csv` ↓ Frozen file identified by SHA-256 hash ↓ Analysis notebook ↓ Audit results and final report

## ✓ 4. Data Quality and Measurement Audit

The following audits check the structure, validity, completeness, duplication, potential anomalies, and possible target leakage in the frozen `day.csv` dataset.

### ✓ 4.1 Schema Audit

The schema audit checks the number of observations and variables, column names, data types, and whether the expected variables are present.

```
import pandas as pd

# Load the frozen dataset
df = pd.read_csv("/content/day.csv")

print("Shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)
```

```
Shape: (731, 16)
```

```
Column names:
['instant', 'dteday', 'season', 'yr', 'mnth', 'holiday', 'weekday', 'workingday', 'weathersit', 'temp', 'atemp', 'hum', 'windspe

Data types:
instant      int64
dteday       object
season       int64
yr           int64
mnth         int64
holiday      int64
weekday      int64
workingday   int64
weathersit    int64
temp         float64
atemp        float64
hum          float64
windspeed    float64
casual       int64
registered   int64
cnt          int64
dtype: object
```

```
schema = pd.DataFrame({
    "column": df.columns,
    "dtype": df.dtypes.astype(str).values,
    "non_null": df.notna().sum().values,
    "unique_values": df.nunique().values
})
```

schema

|    | column     | dtype   | non_null | unique_values |
|----|------------|---------|----------|---------------|
| 0  | instant    | int64   | 731      | 731           |
| 1  | dteday     | object  | 731      | 731           |
| 2  | season     | int64   | 731      | 4             |
| 3  | yr         | int64   | 731      | 2             |
| 4  | mnth       | int64   | 731      | 12            |
| 5  | holiday    | int64   | 731      | 2             |
| 6  | weekday    | int64   | 731      | 7             |
| 7  | workingday | int64   | 731      | 2             |
| 8  | weathersit | int64   | 731      | 3             |
| 9  | temp       | float64 | 731      | 499           |
| 10 | atemp      | float64 | 731      | 690           |
| 11 | hum        | float64 | 731      | 595           |
| 12 | windspeed  | float64 | 731      | 650           |
| 13 | casual     | int64   | 731      | 606           |
| 14 | registered | int64   | 731      | 679           |
| 15 | cnt        | int64   | 731      | 696           |

Schema Audit Result

The frozen `day.csv` file contains 731 observations and 16 variables. All 16 variables are present, and each variable contains 731 non-null observations. The dataset includes integer, floating-point, and date/string fields. The target variable `cnt` is stored as an integer. The date variable `dteday` is stored as an object/string type and contains 731 unique values.

The observed cardinalities are consistent with the documented categorical variables: 4 seasons, 2 years, 12 months, 2 holiday states, 7 weekdays, 2 working-day states, and 3 weather-situation categories.

4.2 Range and Validity Audit

The range audit checks whether categorical and numerical variables contain values outside their expected or documented ranges. Values outside the expected ranges would require investigation before analysis.

```
# Range and validity checks

range_checks = {
    "instant": (df["instant"].min(), df["instant"].max()),
    "season": (df["season"].min(), df["season"].max()),
    "yr": (df["yr"].min(), df["yr"].max()),
    "mnth": (df["mnth"].min(), df["mnth"].max()),
    "holiday": (df["holiday"].min(), df["holiday"].max()),
    "weekday": (df["weekday"].min(), df["weekday"].max()),
    "workingday": (df["workingday"].min(), df["workingday"].max()),
    "weathersit": (df["weathersit"].min(), df["weathersit"].max()),
    "temp": (df["temp"].min(), df["temp"].max()),
    "atemp": (df["atemp"].min(), df["atemp"].max()),
    "hum": (df["hum"].min(), df["hum"].max()),
    "windspeed": (df["windspeed"].min(), df["windspeed"].max()),
    "casual": (df["casual"].min(), df["casual"].max()),
    "registered": (df["registered"].min(), df["registered"].max()),
    "cnt": (df["cnt"].min(), df["cnt"].max())
}

for variable, values in range_checks.items():
    print(f"{variable:12s}: min = {values[0]}, max = {values[1]}")
```

```
instant      : min = 1, max = 731
season       : min = 1, max = 4
yr           : min = 0, max = 1
mnth         : min = 1, max = 12
holiday      : min = 0, max = 1
weekday      : min = 0, max = 6
workingday   : min = 0, max = 1
weathersit    : min = 1, max = 3
temp         : min = 0.0591304, max = 0.861667
atemp        : min = 0.0790696, max = 0.840896
hum          : min = 0.0, max = 0.9725
windspeed    : min = 0.0223917, max = 0.507463
casual       : min = 2, max = 3410
registered   : min = 20, max = 6946
cnt          : min = 22, max = 8714
```

```
# Check expected categorical values

expected_values = {
    "season": {1, 2, 3, 4},
    "yr": {0, 1},
    "mnth": set(range(1, 13)),
    "holiday": {0, 1},
    "weekday": set(range(7)),
    "workingday": {0, 1},
    "weathersit": {1, 2, 3, 4}
}

for column, expected in expected_values.items():
    observed = set(df[column].unique())
    unexpected = observed - expected

    print(f"{column}:")
    print("  Observed values:", sorted(observed))
    print("  Unexpected values:", sorted(unexpected))
```

```
season:
  Observed values: [np.int64(1), np.int64(2), np.int64(3), np.int64(4)]
  Unexpected values: []
yr:
  Observed values: [np.int64(0), np.int64(1)]
  Unexpected values: []
mnth:
  Observed values: [np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8), np.i
  Unexpected values: []
holiday:
  Observed values: [np.int64(0), np.int64(1)]
  Unexpected values: []
weekday:
  Observed values: [np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6)]
  Unexpected values: []
workingday:
  Observed values: [np.int64(0), np.int64(1)]
  Unexpected values: []
weathersit:
```

Observed values: [np.int64(1), np.int64(2), np.int64(3)]

Unexpected values: []

Range and Validity Audit Result

All categorical variables contain values within their expected ranges, with no unexpected category codes detected. `season` contains values 1–4, `yr` contains 0–1, `mnth` contains 1–12, `holiday` and `workingday` contain 0–1, `weekday` contains 0–6, and `weathersit` contains 1–3.

The normalized weather variables also fall within bounded ranges in the observed data: `temp` ranges from 0.0591 to 0.8617, `atemp` from 0.0791 to 0.8409, `hum` from 0.0000 to 0.9725, and `windspeed` from 0.0224 to 0.5075. Rental counts are positive, with `casual` ranging from 2 to 3,410, `registered` from 20 to 6,946, and `cnt` from 22 to 8,714.

No range violation was identified in this audit.

4.3 Missingness Audit

The missingness audit checks the number and proportion of missing values in each variable. Missing observations can affect the completeness and reproducibility of the analysis.

```
# Missingness audit

missing_count = df.isna().sum()
missing_percent = (df.isna().mean() * 100).round(2)

missing_audit = pd.DataFrame({
    "missing_count": missing_count,
    "missing_percent": missing_percent
})

missing_audit
```

|            | missing_count | missing_percent |
|------------|---------------|-----------------|
| instant    | 0             | 0.0             |
| dteday     | 0             | 0.0             |
| season     | 0             | 0.0             |
| yr         | 0             | 0.0             |
| mnth       | 0             | 0.0             |
| holiday    | 0             | 0.0             |
| weekday    | 0             | 0.0             |
| workingday | 0             | 0.0             |
| weathersit | 0             | 0.0             |
| temp       | 0             | 0.0             |
| atemp      | 0             | 0.0             |
| hum        | 0             | 0.0             |
| windspeed  | 0             | 0.0             |
| casual     | 0             | 0.0             |
| registered | 0             | 0.0             |
| cnt        | 0             | 0.0             |

```
# Overall missingness

total_missing = df.isna().sum().sum()

print("Total missing values:", total_missing)

if total_missing == 0:
    print("Result: No missing values detected.")
else:
    print("Result: Missing values detected.")
```

```
Total missing values: 0
Result: No missing values detected.
```

## Missingness Audit Result

No missing values were detected in the frozen dataset. All 16 variables contain complete observations across all 731 daily records, corresponding to 0.0% missingness for every variable. Therefore, no missing-value imputation or row removal is required for the subsequent audit and analysis.

### 4.4 Duplicate Audit

The duplicate audit checks for completely duplicated records and repeated calendar dates. Because the unit of analysis is one calendar day, each date is expected to occur once in the daily dataset.

```
# Duplicate audit

duplicate_rows = df.duplicated().sum()
duplicate_dates = df["dteday"].duplicated().sum()

print("Completely duplicated rows:", duplicate_rows)
print("Repeated dates:", duplicate_dates)
```

```
Completely duplicated rows: 0
Repeated dates: 0
```

```
# Display duplicate dates, if any

duplicate_date_values = df.loc[
    df["dteday"].duplicated(keep=False),
    "dteday"
].sort_values()

if duplicate_date_values.empty:
    print("No duplicate dates detected.")
else:
    print("Duplicate dates:")
    print(duplicate_date_values.tolist())
```

```
No duplicate dates detected.
```

## Duplicate Audit Result

No completely duplicated rows and no repeated calendar dates were detected. This is consistent with the defined unit of analysis of one calendar day, with each of the 731 daily observations representing a unique date.

### 4.5 Anomaly Audit

The anomaly audit identifies observations that are unusual relative to the observed distribution or that may violate logical relationships between variables. An anomaly is treated as a record requiring investigation rather than automatically being considered an error.

```
# IQR-based anomaly screening for numerical variables

numeric_columns = [
    "temp", "atemp", "hum", "windspeed",
    "casual", "registered", "cnt"
]
```

```

anomaly_summary = []

for column in numeric_columns:
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    n_anomalies = ((df[column] < lower_bound) |
                    (df[column] > upper_bound)).sum()

    anomaly_summary.append({
        "variable": column,
        "Q1": Q1,
        "Q3": Q3,
        "lower_bound": lower_bound,
        "upper_bound": upper_bound,
        "anomaly_count": n_anomalies
    })

anomaly_summary = pd.DataFrame(anomaly_summary)

anomaly_summary

```

|   | variable   | Q1          | Q3          | lower_bound  | upper_bound  | anomaly_count |
|---|------------|-------------|-------------|--------------|--------------|---------------|
| 0 | temp       | 0.337083    | 0.655417    | -0.140416    | 1.132916     | 0             |
| 1 | atemp      | 0.337842    | 0.608602    | -0.068297    | 1.014741     | 0             |
| 2 | hum        | 0.520000    | 0.730209    | 0.204687     | 1.045521     | 2             |
| 3 | windspeed  | 0.134950    | 0.233214    | -0.012447    | 0.380611     | 13            |
| 4 | casual     | 315.500000  | 1096.000000 | -855.250000  | 2266.750000  | 44            |
| 5 | registered | 2497.000000 | 4776.500000 | -922.250000  | 8195.750000  | 0             |
| 6 | cnt        | 3152.000000 | 5956.000000 | -1054.000000 | 10162.000000 | 0             |

```

# Logical consistency check for rental counts

count_mismatch = df["casual"] + df["registered"] != df["cnt"]

print("Rows where casual + registered != cnt:", count_mismatch.sum())

if count_mismatch.sum() == 0:
    print("Result: casual + registered equals cnt for every observation.")
else:
    print("Result: Some observations have inconsistent rental totals.")

```

Rows where casual + registered != cnt: 0  
 Result: casual + registered equals cnt for every observation.

```

# Basic validity checks for rental counts

print("Negative casual counts:", (df["casual"] < 0).sum())
print("Negative registered counts:", (df["registered"] < 0).sum())
print("Negative total counts:", (df["cnt"] < 0).sum())

```

Negative casual counts: 0  
 Negative registered counts: 0  
 Negative total counts: 0

## Anomaly Audit Result

The IQR screening identified no potential anomalies for `temp`, `atemp`, `registered`, or `cnt`. It identified 2 observations for `hum`, 13 for `windspeed`, and 44 for `casual` as unusually high or low relative to the IQR-based thresholds.

These IQR flags are treated as potential distributional anomalies rather than data errors. Unusually high rental counts or weather measurements can represent legitimate days in the observed system. No negative values were found for `casual`, `registered`, or `cnt`. In addition, `casual + registered` equals `cnt` for all 731 observations, indicating internal consistency of the rental-count variables.

No observation was removed based solely on the IQR screening.

### 4.6 Leakage Audit

The leakage audit checks whether any available variables directly contain the target or are constructed from the target. The target for this analysis is `cnt`, the total number of daily rentals.

```
# Check whether rental components reconstruct the target

reconstructed_cnt = df["casual"] + df["registered"]

print("Does casual + registered equal cnt for all rows?",
      (reconstructed_cnt == df["cnt"]).all())

print("Number of mismatches:",
      (reconstructed_cnt != df["cnt"]).sum())
```

```
Does casual + registered equal cnt for all rows? True
Number of mismatches: 0
```

```
# Correlation of numeric variables with the target

numeric_df = df.select_dtypes(include="number")

correlation_with_cnt = (
    numeric_df.corr()["cnt"]
    .sort_values(ascending=False)
)

correlation_with_cnt
```

|                   | cnt       |
|-------------------|-----------|
| <b>cnt</b>        | 1.000000  |
| <b>registered</b> | 0.945517  |
| <b>casual</b>     | 0.672804  |
| <b>atemp</b>      | 0.631066  |
| <b>instant</b>    | 0.628830  |
| <b>temp</b>       | 0.627494  |
| <b>yr</b>         | 0.566710  |
| <b>season</b>     | 0.406100  |
| <b>mnth</b>       | 0.279977  |
| <b>weekday</b>    | 0.067443  |
| <b>workingday</b> | 0.061156  |
| <b>holiday</b>    | -0.068348 |
| <b>hum</b>        | -0.100659 |
| <b>windspeed</b>  | -0.234545 |
| <b>weathersit</b> | -0.297391 |

```
dtype: float64
```

```
# Variables excluded because they directly reconstruct the target
```

```
leakage_variables = ["casual", "registered"]

analysis_features = [
    "season",
    "yr",
    "mnth",
    "holiday",
    "weekday",
    "workingday",
    "weathersit",
    "temp",
    "atemp",
    "hum",
    "windspeed"
]

print("Target:", "cnt")
print("Excluded leakage variables:", leakage_variables)
print("Analysis features:", analysis_features)
```

```
Target: cnt
Excluded leakage variables: ['casual', 'registered']
Analysis features: ['season', 'yr', 'mnth', 'holiday', 'weekday', 'workingday', 'weathersit', 'temp', 'atemp', 'hum', 'windspeed']
```

## Leakage Audit Result

The audit confirmed that `casual + registered` equals `cnt` for all 731 observations, with zero mismatches. Therefore, `casual` and `registered` are direct components of the target and would create target leakage if used as predictors of `cnt`.

The correlation audit also shows a very high correlation between `registered` and `cnt` ( $r = 0.946$ ) and a substantial correlation between `casual` and `cnt` ( $r = 0.673$ ). These correlations are not interpreted as independent predictive evidence because the two variables directly compose the target.

For the subsequent analysis, `casual` and `registered` are excluded as predictors. The analysis feature set consists of calendar, weather, and normalized weather variables listed above. This prevents direct target leakage.

## 5. Bias, Measurement, and Claim Boundaries

### 5.1 Bias/Measurement Counterexample

A measurement limitation is that the dataset records weather variables using normalized measurements rather than direct physical units in the analysis file. In addition, daily rental demand is an aggregate system-level count and does not record individual users' reasons for renting or not renting a bicycle.

For example, two days with similar recorded temperature and humidity can have different rental totals because other factors affecting demand—such as events, traffic conditions, accessibility, or individual travel decisions—may differ but are not fully represented in the dataset. Therefore, the observed weather-demand association should not be interpreted as a complete explanation of daily rental variation.

### 5.2 Prohibited Claim

The analysis will not claim that weather conditions cause changes in bike-rental demand. The dataset is observational, so the analysis can describe associations between observed weather conditions and daily rental demand but cannot establish a causal effect of weather from these data alone.

A prohibited claim is:

"Higher temperatures cause people to rent more bicycles."

A supported formulation is:

"Daily bike-rental demand is associated with observed temperature and other weather conditions in the 2011–2012 Capital Bikeshare data."

## 6. Data Card

### Dataset

UCI Bike Sharing Dataset — daily bike-sharing data (`day.csv`).

### Source

UCI Machine Learning Repository.

### License

CC BY 4.0.

### Study period

2011–2012.

### Sample

731 daily observations.

### Unit of analysis

One calendar day.

### Target

`cnt`, the total number of bike rentals per day.

### Intended use

The dataset is used to examine associations between observed weather conditions and daily bike-rental demand within the observed Capital Bikeshare data from 2011–2012.

### Out-of-scope use

The dataset should not be used to establish that weather conditions causally change bike-rental demand. It should also not be treated as representative of all bike-sharing systems, cities, or time periods.

### Known limitations

- The observations are daily aggregates rather than individual rental records.
- The dataset does not capture every factor that may influence daily rental demand.
- Weather variables are represented as normalized measurements in `day.csv`.
- The data cover 2011–2012 and therefore may not represent current bike-sharing behavior.
- `casual` and `registered` are components of `cnt` and are excluded from predictor analysis to avoid target leakage.

### Frozen data version

File: `day.csv`

File size: 57,569 bytes

SHA-256: `a6bcf826782d3c0fbfdbcbeead17cd0884185a0dafa8ff10cd48a874ee7ba18be`

## 7. Data Dictionary

| Variable                | Type        | Description                 | Role                  |
|-------------------------|-------------|-----------------------------|-----------------------|
| <code>instant</code>    | Integer     | Record index                | Identifier/time index |
| <code>dteday</code>     | Date/String | Calendar date               | Date                  |
| <code>season</code>     | Integer     | Season category (1–4)       | Covariate             |
| <code>yr</code>         | Integer     | Year indicator (0–1)        | Covariate             |
| <code>mnth</code>       | Integer     | Month (1–12)                | Covariate             |
| <code>holiday</code>    | Integer     | Holiday indicator (0/1)     | Covariate             |
| <code>weekday</code>    | Integer     | Day of week (0–6)           | Covariate             |
| <code>workingday</code> | Integer     | Working-day indicator (0/1) | Covariate             |
| <code>weathersit</code> | Integer     | Weather situation category  | Weather covariate     |

| Variable   | Type    | Description                    | Role                       |
|------------|---------|--------------------------------|----------------------------|
| temp       | Numeric | Normalized temperature         | Main weather variable      |
| atemp      | Numeric | Normalized feeling temperature | Weather variable           |
| hum        | Numeric | Normalized humidity            | Weather variable           |
| windspeed  | Numeric | Normalized wind speed          | Weather variable           |
| casual     | Integer | Number of casual rentals       | Target component; excluded |
| registered | Integer | Number of registered rentals   | Target component; excluded |
| cnt        | Integer | Total daily bike rentals       | Target                     |

8. Exploratory Analysis

The exploratory analysis summarizes daily rental demand and examines its observed relationship with weather variables. The analysis is descriptive and associational rather than causal.

```
# Descriptive statistics for target and weather variables
```

```
weather_target = [
    "temp",
    "atemp",
    "hum",
    "windspeed",
    "cnt"
]

df[weather_target].describe().T
```

|           | count | mean        | std         | min       | 25%         | 50%         | 75%         | max         |
|-----------|-------|-------------|-------------|-----------|-------------|-------------|-------------|-------------|
| temp      | 731.0 | 0.495385    | 0.183051    | 0.059130  | 0.337083    | 0.498333    | 0.655417    | 0.861667    |
| atemp     | 731.0 | 0.474354    | 0.162961    | 0.079070  | 0.337842    | 0.486733    | 0.608602    | 0.840896    |
| hum       | 731.0 | 0.627894    | 0.142429    | 0.000000  | 0.520000    | 0.626667    | 0.730209    | 0.972500    |
| windspeed | 731.0 | 0.190486    | 0.077498    | 0.022392  | 0.134950    | 0.180975    | 0.233214    | 0.507463    |
| cnt       | 731.0 | 4504.348837 | 1937.211452 | 22.000000 | 3152.000000 | 4548.000000 | 5956.000000 | 8714.000000 |

```
# Correlation between weather variables and daily rental demand
```

```
df[weather_target].corr()["cnt"].sort_values(ascending=False)
```

|           | cnt       |
|-----------|-----------|
| cnt       | 1.000000  |
| atemp     | 0.631066  |
| temp      | 0.627494  |
| hum       | -0.100659 |
| windspeed | -0.234545 |

```
dtype: float64
```

8.1 Daily Rentals and Temperature

The following scatter plot shows the observed association between normalized temperature and daily rental demand. The plot is descriptive and does not establish causation.

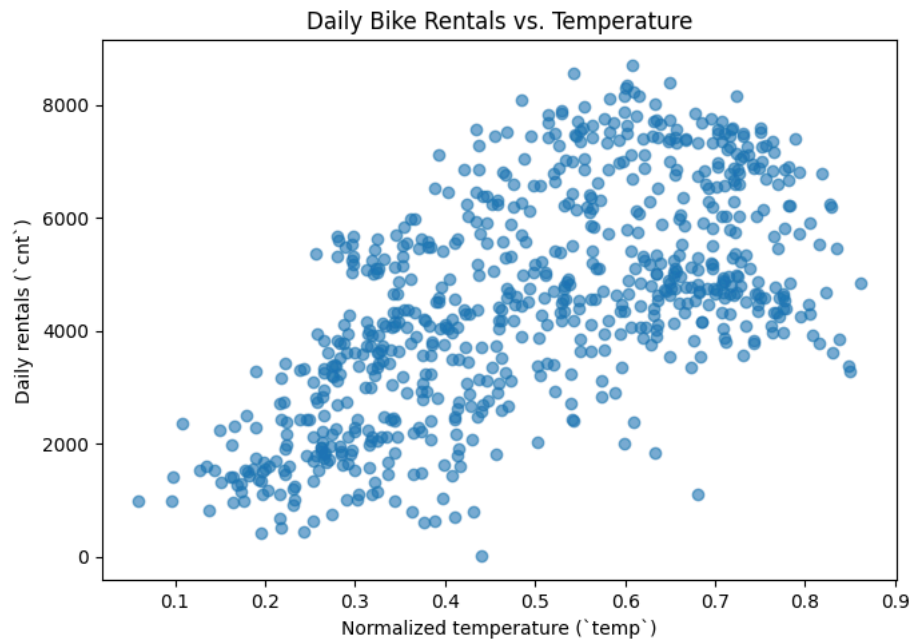
```
import matplotlib.pyplot as plt

plt.figure(figsize=(7, 5))

plt.scatter(df["temp"], df["cnt"], alpha=0.6)

plt.xlabel("Normalized temperature (`temp`)")
plt.ylabel("Daily rentals (`cnt`)")
plt.title("Daily Bike Rentals vs. Temperature")
```

```
plt.tight_layout()
plt.show()
```



## 8.2 Daily Rentals by Weather Situation

The following analysis compares daily rental demand across the observed weather-situation categories. The comparison describes differences in rental demand across weather categories and does not establish causation.

```
# Summary of daily rentals by weather situation

weather_summary = (
    df.groupby("weathersit")["cnt"]
    .agg(["count", "mean", "median", "std", "min", "max"])
    .round(2)
)

weather_summary
```

|            | count | mean    | median | std     | min | max  |
|------------|-------|---------|--------|---------|-----|------|
| weathersit |       |         |        |         |     |      |
| 1          | 463   | 4876.79 | 4844.0 | 1879.48 | 431 | 8714 |
| 2          | 247   | 4035.86 | 4040.0 | 1809.11 | 605 | 8362 |
| 3          | 21    | 1803.29 | 1817.0 | 1240.28 | 22  | 4639 |

```
# Number of observations in each weather category

weather_counts = df["weathersit"].value_counts().sort_index()

weather_counts
```

|            | count |
|------------|-------|
| weathersit |       |
| 1          | 463   |
| 2          | 247   |
| 3          | 21    |

**dtype:** int64

```
import matplotlib.pyplot as plt
```

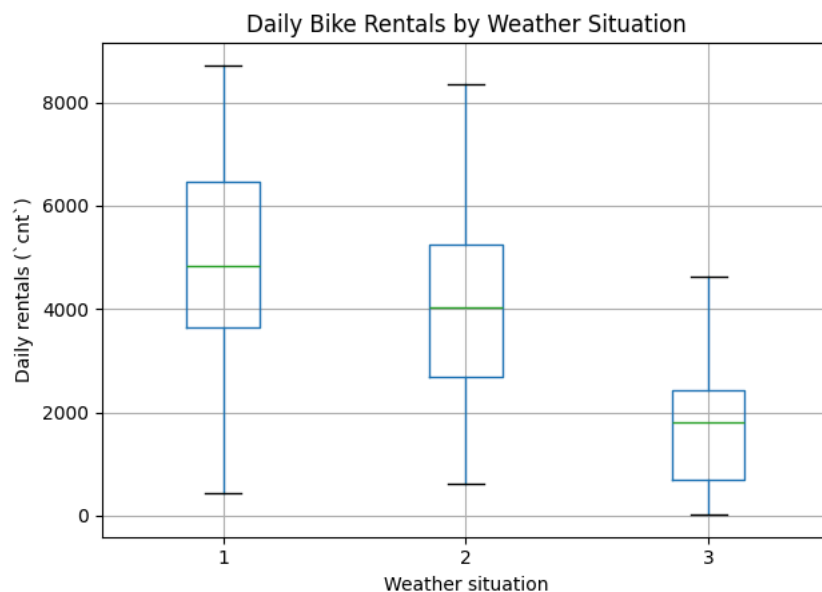
```
plt.figure(figsize=(7, 5))

df.boxplot(column="cnt", by="weathersit")

plt.xlabel("Weather situation")
plt.ylabel("Daily rentals (`cnt`)")
plt.title("Daily Bike Rentals by Weather Situation")
plt.suptitle("")

plt.tight_layout()
plt.show()
```

<Figure size 700x500 with 0 Axes>



### 8.3 Temperature and Daily Rental Demand

The scatter plot is used to examine the form of the observed relationship between temperature and daily rental demand. A visual trend does not by itself establish a causal relationship.

```
import numpy as np
import matplotlib.pyplot as plt

x = df["temp"]
y = df["cnt"]

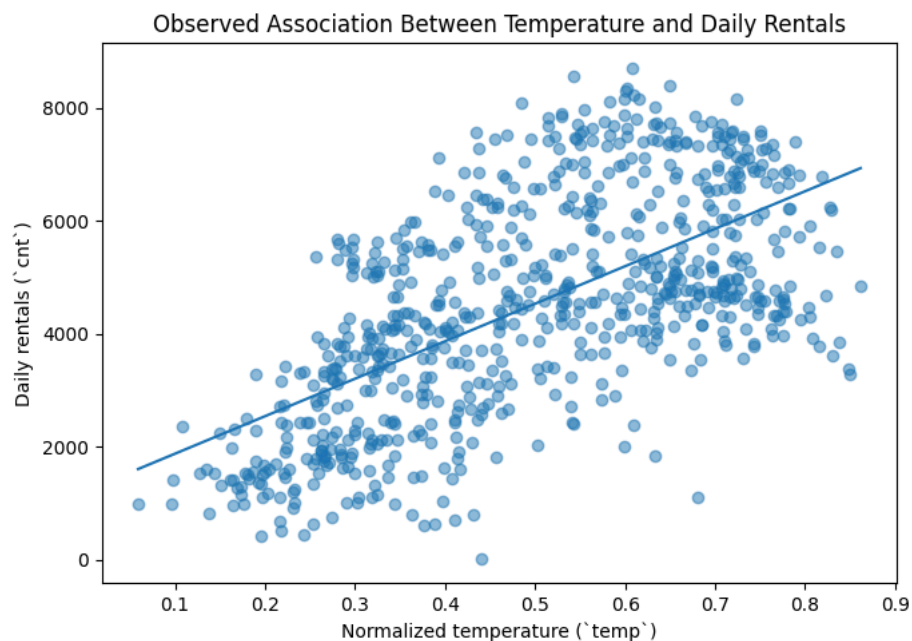
# Simple descriptive linear trend
slope, intercept = np.polyfit(x, y, 1)
x_line = np.linspace(x.min(), x.max(), 100)
y_line = slope * x_line + intercept

plt.figure(figsize=(7, 5))
plt.scatter(x, y, alpha=0.5)
plt.plot(x_line, y_line)

plt.xlabel("Normalized temperature (`temp`)")
plt.ylabel("Daily rentals (`cnt`)")
plt.title("Observed Association Between Temperature and Daily Rentals")

plt.tight_layout()
plt.show()

print(f"Descriptive linear slope: {slope:.2f}")
```



## 8.3 Predictor Correlation and Redundancy Check

```
predictors = [
    "season", "yr", "mnth", "holiday", "weekday", "workingday",
    "weathersit", "temp", "atemp", "hum", "windspeed"
]
```

```
predictor_corr = df[predictors].corr().round(3)
```

```
predictor_corr
```

|            | season | yr     | mnth   | holiday | weekday | workingday | weathersit | temp   | atemp  | hum    | windspeed |
|------------|--------|--------|--------|---------|---------|------------|------------|--------|--------|--------|-----------|
| season     | 1.000  | -0.002 | 0.831  | -0.011  | -0.003  | 0.012      | 0.019      | 0.334  | 0.343  | 0.205  | -0.229    |
| yr         | -0.002 | 1.000  | -0.002 | 0.008   | -0.005  | -0.002     | -0.049     | 0.048  | 0.046  | -0.111 | -0.012    |
| mnth       | 0.831  | -0.002 | 1.000  | 0.019   | 0.010   | -0.006     | 0.044      | 0.220  | 0.227  | 0.222  | -0.208    |
| holiday    | -0.011 | 0.008  | 0.019  | 1.000   | -0.102  | -0.253     | -0.035     | -0.029 | -0.033 | -0.016 | 0.006     |
| weekday    | -0.003 | -0.005 | 0.010  | -0.102  | 1.000   | 0.036      | 0.031      | -0.000 | -0.008 | -0.052 | 0.014     |
| workingday | 0.012  | -0.002 | -0.006 | -0.253  | 0.036   | 1.000      | 0.061      | 0.053  | 0.052  | 0.024  | -0.019    |
| weathersit | 0.019  | -0.049 | 0.044  | -0.035  | 0.031   | 0.061      | 1.000      | -0.121 | -0.122 | 0.591  | 0.040     |
| temp       | 0.334  | 0.048  | 0.220  | -0.029  | -0.000  | 0.053      | -0.121     | 1.000  | 0.992  | 0.127  | -0.158    |
| atemp      | 0.343  | 0.046  | 0.227  | -0.033  | -0.008  | 0.052      | -0.122     | 0.992  | 1.000  | 0.140  | -0.184    |
| hum        | 0.205  | -0.111 | 0.222  | -0.016  | -0.052  | 0.024      | 0.591      | 0.127  | 0.140  | 1.000  | -0.248    |
| windspeed  | -0.229 | -0.012 | -0.208 | 0.006   | 0.014   | -0.019     | 0.040      | -0.158 | -0.184 | -0.248 | 1.000     |

```
# Show predictor pairs with absolute correlation >= 0.50
```

```
corr_pairs = (
    predictor_corr.where(
        np.triu(np.ones(predictor_corr.shape), k=1).astype(bool)
    )
    .stack()
    .reset_index()
)

corr_pairs.columns = ["Variable 1", "Variable 2", "Correlation"]
corr_pairs["Absolute correlation"] = corr_pairs["Correlation"].abs()

corr_pairs = (
    corr_pairs[corr_pairs["Absolute correlation"] >= 0.50]
    .sort_values("Absolute correlation", ascending=False)
)
```

```
)
corr_pairs
```

|    | Variable 1 | Variable 2 | Correlation | Absolute correlation |
|----|------------|------------|-------------|----------------------|
| 49 | temp       | atemp      | 0.992       | 0.992                |
| 1  | season     | mnth       | 0.831       | 0.831                |
| 47 | weathersit | hum        | 0.591       | 0.591                |

```
## 8.4 Target Association by Weather Variable
```

```
weather_associations = (
    df[["temp", "atemp", "hum", "windspeed", "cnt"]]
    .corr()["cnt"]
    .drop("cnt")
    .sort_values(ascending=False)
    .round(3)
)

weather_associations
```

|                  | cnt    |
|------------------|--------|
| <b>atemp</b>     | 0.631  |
| <b>temp</b>      | 0.627  |
| <b>hum</b>       | -0.101 |
| <b>windspeed</b> | -0.235 |

```
dtype: float64
```

```
## 9. Main Association Model: Analysis Dataset
```

```
analysis_vars = [
    "cnt",
    "temp",
    "hum",
    "windspeed",
    "weathersit",
    "yr",
    "mnth",
    "holiday",
    "weekday",
    "workingday"
]

analysis_df = df[analysis_vars].copy()

print("Analysis dataset shape:", analysis_df.shape)
print("\nAnalysis variables:")
print(analysis_df.columns.tolist())
print("\nMissing values:")
print(analysis_df.isna().sum())
```

```
Analysis dataset shape: (731, 10)
```

```
Analysis variables:
['cnt', 'temp', 'hum', 'windspeed', 'weathersit', 'yr', 'mnth', 'holiday', 'weekday', 'workingday']
```

```
Missing values:
cnt      0
temp     0
hum      0
windspeed 0
weathersit 0
yr        0
mnth      0
holiday   0
weekday   0
workingday 0
dtype: int64
```

## ## 9.1 Multiple Linear Regression

```
import statsmodels.formula.api as smf
```

```
model = smf.ols(
    formula="""
    cnt ~ temp + hum + windspeed
    + C(weathersit)
    + C(yr)
    + C(mnth)
    + C(holiday)
    + C(weekday)
    + C(workingday)
    """,
    data=analysis_df
).fit(cov_type="HC3")

print(model.summary())
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          cnt      R-squared:                0.829
Model:                  OLS      Adj. R-squared:           0.823
Method:                 Least Squares      F-statistic:       190.1
Date:                  Tue, 29 Sep 2026      Prob (F-statistic):   4.75e-289
Time:                  09:26:04      Log-Likelihood:      -5924.1
No. Observations:      731      AIC:                  1.190e+04
Df Residuals:          706      BIC:                  1.201e+04
Df Model:               24
Covariance Type:       HC3
=====

```

|                    | coef       | std err | z      | P> z  | [0.025    | 0.975]    |
|--------------------|------------|---------|--------|-------|-----------|-----------|
| Intercept          | 1561.8155  | 251.340 | 6.214  | 0.000 | 1069.199  | 2054.432  |
| C(weathersit)[T.2] | -456.0489  | 80.313  | -5.678 | 0.000 | -613.460  | -298.638  |
| C(weathersit)[T.3] | -1981.5207 | 310.849 | -6.375 | 0.000 | -2590.774 | -1372.267 |
| C(yr)[T.1]         | 2016.4004  | 66.375  | 30.379 | 0.000 | 1886.307  | 2146.493  |
| C(mnth)[T.2]       | 131.2283   | 99.842  | 1.314  | 0.189 | -64.459   | 326.916   |
| C(mnth)[T.3]       | 838.6858   | 144.259 | 5.814  | 0.000 | 555.943   | 1121.429  |
| C(mnth)[T.4]       | 1321.4447  | 168.378 | 7.848  | 0.000 | 991.429   | 1651.460  |
| C(mnth)[T.5]       | 1546.7591  | 212.963 | 7.263  | 0.000 | 1129.359  | 1964.159  |
| C(mnth)[T.6]       | 1285.5543  | 267.570 | 4.805  | 0.000 | 761.127   | 1809.981  |
| C(mnth)[T.7]       | 751.9751   | 300.926 | 2.499  | 0.012 | 162.171   | 1341.779  |
| C(mnth)[T.8]       | 1150.1176  | 270.739 | 4.248  | 0.000 | 619.478   | 1680.757  |
| C(mnth)[T.9]       | 1939.9776  | 234.028 | 8.290  | 0.000 | 1481.291  | 2398.664  |
| C(mnth)[T.10]      | 2046.4923  | 211.087 | 9.695  | 0.000 | 1632.770  | 2460.215  |
| C(mnth)[T.11]      | 1443.2020  | 155.859 | 9.260  | 0.000 | 1137.724  | 1748.680  |
| C(mnth)[T.12]      | 905.8915   | 179.890 | 5.036  | 0.000 | 553.313   | 1258.470  |
| C(holiday)[T.1]    | -384.6118  | 263.629 | -1.459 | 0.145 | -901.315  | 132.091   |
| C(weekday)[T.1]    | -118.3443  | 78.628  | -1.505 | 0.132 | -272.452  | 35.763    |
| C(weekday)[T.2]    | -36.7464   | 92.883  | -0.396 | 0.692 | -218.794  | 145.301   |
| C(weekday)[T.3]    | 23.9827    | 80.102  | 0.299  | 0.765 | -133.015  | 180.980   |
| C(weekday)[T.4]    | 35.5737    | 92.212  | 0.386  | 0.700 | -145.159  | 216.307   |
| C(weekday)[T.5]    | 64.8124    | 85.025  | 0.762  | 0.446 | -101.834  | 231.459   |
| C(weekday)[T.6]    | 429.8173   | 131.566 | 3.267  | 0.001 | 171.952   | 687.683   |
| C(workingday)[T.1] | 353.8899   | 79.779  | 4.436  | 0.000 | 197.526   | 510.254   |
| temp               | 4629.9235  | 530.543 | 8.727  | 0.000 | 3590.078  | 5669.769  |
| hum                | -1477.5290 | 329.385 | -4.486 | 0.000 | -2123.112 | -831.946  |
| windspeed          | -3286.6508 | 501.515 | -6.553 | 0.000 | -4269.603 | -2303.699 |

```

=====
Omnibus:                 113.944      Durbin-Watson:           1.069
Prob(Omnibus):            0.000      Jarque-Bera (JB):        265.116
Skew:                     -0.843      Prob(JB):                 2.70e-58
Kurtosis:                  5.421      Cond. No.                  2.12e+15
=====

```

## Notes:

[1] Standard Errors are heteroscedasticity robust (HC3)

[2] The smallest eigenvalue is 4.56e-28. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

/tmp/ipykernel\_46970/679520488.py:16: SingularMatrixWarning: The design matrix is rank-deficient. The model parameters are not u  
) .fit(cov\_type="HC3")

/usr/local/lib/python3.13/dist-packages/statsmodels/base/model.py:1988: ValueWarning: covariance of constraints does not have fu  
res = self.wald\_test()

## ## 9.2 Revised Association Model

```
model_revised = smf.ols(
    formula="""
    cnt ~ temp + hum + windspeed
    + C(weathersit)

```

```

+ C(yr)
+ C(mnth)
+ C(weekday)
+ C(workingday)
""",
data=analysis_df
).fit(cov_type="HC3")

print(model_revised.summary())

```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          cnt      R-squared:                0.829
Model:                  OLS      Adj. R-squared:            0.823
Method:                 Least Squares      F-statistic:        190.1
Date:                   Tue, 29 Sep 2026      Prob (F-statistic):    4.75e-289
Time:                   09:26:04      Log-Likelihood:       -5924.1
No. Observations:       731      AIC:                  1.190e+04
Df Residuals:           706      BIC:                  1.201e+04
Df Model:                24
Covariance Type:        HC3
=====

```

|                    | coef       | std err | z      | P> z  | [0.025    | 0.975]    |
|--------------------|------------|---------|--------|-------|-----------|-----------|
| Intercept          | 1561.8155  | 251.340 | 6.214  | 0.000 | 1069.199  | 2054.432  |
| C(weathersit)[T.2] | -456.0489  | 80.313  | -5.678 | 0.000 | -613.460  | -298.638  |
| C(weathersit)[T.3] | -1981.5207 | 310.849 | -6.375 | 0.000 | -2590.774 | -1372.267 |
| C(yr)[T.1]         | 2016.4004  | 66.375  | 30.379 | 0.000 | 1886.307  | 2146.493  |
| C(mnth)[T.2]       | 131.2283   | 99.842  | 1.314  | 0.189 | -64.459   | 326.916   |
| C(mnth)[T.3]       | 838.6858   | 144.259 | 5.814  | 0.000 | 555.943   | 1121.429  |
| C(mnth)[T.4]       | 1321.4447  | 168.378 | 7.848  | 0.000 | 991.429   | 1651.460  |
| C(mnth)[T.5]       | 1546.7591  | 212.963 | 7.263  | 0.000 | 1129.359  | 1964.159  |
| C(mnth)[T.6]       | 1285.5543  | 267.570 | 4.805  | 0.000 | 761.127   | 1809.981  |
| C(mnth)[T.7]       | 751.9751   | 300.926 | 2.499  | 0.012 | 162.171   | 1341.779  |
| C(mnth)[T.8]       | 1150.1176  | 270.739 | 4.248  | 0.000 | 619.478   | 1680.757  |
| C(mnth)[T.9]       | 1939.9776  | 234.028 | 8.290  | 0.000 | 1481.291  | 2398.664  |
| C(mnth)[T.10]      | 2046.4923  | 211.087 | 9.695  | 0.000 | 1632.770  | 2460.215  |
| C(mnth)[T.11]      | 1443.2020  | 155.859 | 9.260  | 0.000 | 1137.724  | 1748.680  |
| C(mnth)[T.12]      | 905.8915   | 179.890 | 5.036  | 0.000 | 553.313   | 1258.470  |
| C(weekday)[T.1]    | -502.9562  | 291.414 | -1.726 | 0.084 | -1074.118 | 68.205    |
| C(weekday)[T.2]    | -421.3583  | 331.229 | -1.272 | 0.203 | -1070.554 | 227.838   |
| C(weekday)[T.3]    | -360.6291  | 319.735 | -1.128 | 0.259 | -987.298  | 266.040   |
| C(weekday)[T.4]    | -349.0381  | 334.125 | -1.045 | 0.296 | -1003.911 | 305.835   |
| C(weekday)[T.5]    | -319.7995  | 316.087 | -1.012 | 0.312 | -939.319  | 299.720   |
| C(weekday)[T.6]    | 429.8173   | 131.566 | 3.267  | 0.001 | 171.952   | 687.683   |
| C(workingday)[T.1] | 738.5017   | 301.509 | 2.449  | 0.014 | 147.554   | 1329.449  |
| temp               | 4629.9235  | 530.543 | 8.727  | 0.000 | 3590.078  | 5669.769  |
| hum                | -1477.5290 | 329.385 | -4.486 | 0.000 | -2123.112 | -831.946  |
| windspeed          | -3286.6508 | 501.515 | -6.553 | 0.000 | -4269.603 | -2303.699 |

```

=====
Omnibus:                 113.944      Durbin-Watson:          1.069
Prob(Omnibus):            0.000      Jarque-Bera (JB):        265.116
Skew:                     -0.843      Prob(JB):                2.70e-58
Kurtosis:                 5.421      Cond. No.                 37.7
=====

```

Notes:

[1] Standard Errors are heteroscedasticity robust (HC3)

```
print(model_revised.summary())
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          cnt      R-squared:                0.829
Model:                  OLS      Adj. R-squared:            0.823
Method:                 Least Squares      F-statistic:        190.1
Date:                   Tue, 29 Sep 2026      Prob (F-statistic):    4.75e-289
Time:                   09:26:05      Log-Likelihood:       -5924.1
No. Observations:       731      AIC:                  1.190e+04
Df Residuals:           706      BIC:                  1.201e+04
Df Model:                24
Covariance Type:        HC3
=====

```

|                    | coef       | std err | z      | P> z  | [0.025    | 0.975]    |
|--------------------|------------|---------|--------|-------|-----------|-----------|
| Intercept          | 1561.8155  | 251.340 | 6.214  | 0.000 | 1069.199  | 2054.432  |
| C(weathersit)[T.2] | -456.0489  | 80.313  | -5.678 | 0.000 | -613.460  | -298.638  |
| C(weathersit)[T.3] | -1981.5207 | 310.849 | -6.375 | 0.000 | -2590.774 | -1372.267 |
| C(yr)[T.1]         | 2016.4004  | 66.375  | 30.379 | 0.000 | 1886.307  | 2146.493  |
| C(mnth)[T.2]       | 131.2283   | 99.842  | 1.314  | 0.189 | -64.459   | 326.916   |
| C(mnth)[T.3]       | 838.6858   | 144.259 | 5.814  | 0.000 | 555.943   | 1121.429  |
| C(mnth)[T.4]       | 1321.4447  | 168.378 | 7.848  | 0.000 | 991.429   | 1651.460  |
| C(mnth)[T.5]       | 1546.7591  | 212.963 | 7.263  | 0.000 | 1129.359  | 1964.159  |

|                    |            |         |        |       |           |           |
|--------------------|------------|---------|--------|-------|-----------|-----------|
| C(mnth)[T.6]       | 1285.5543  | 267.570 | 4.805  | 0.000 | 761.127   | 1809.981  |
| C(mnth)[T.7]       | 751.9751   | 300.926 | 2.499  | 0.012 | 162.171   | 1341.779  |
| C(mnth)[T.8]       | 1150.1176  | 270.739 | 4.248  | 0.000 | 619.478   | 1680.757  |
| C(mnth)[T.9]       | 1939.9776  | 234.028 | 8.290  | 0.000 | 1481.291  | 2398.664  |
| C(mnth)[T.10]      | 2046.4923  | 211.087 | 9.695  | 0.000 | 1632.770  | 2460.215  |
| C(mnth)[T.11]      | 1443.2020  | 155.859 | 9.260  | 0.000 | 1137.724  | 1748.680  |
| C(mnth)[T.12]      | 905.8915   | 179.890 | 5.036  | 0.000 | 553.313   | 1258.470  |
| C(weekday)[T.1]    | -502.9562  | 291.414 | -1.726 | 0.084 | -1074.118 | 68.205    |
| C(weekday)[T.2]    | -421.3583  | 331.229 | -1.272 | 0.203 | -1070.554 | 227.838   |
| C(weekday)[T.3]    | -360.6291  | 319.735 | -1.128 | 0.259 | -987.298  | 266.040   |
| C(weekday)[T.4]    | -349.0381  | 334.125 | -1.045 | 0.296 | -1003.911 | 305.835   |
| C(weekday)[T.5]    | -319.7995  | 316.087 | -1.012 | 0.312 | -939.319  | 299.720   |
| C(weekday)[T.6]    | 429.8173   | 131.566 | 3.267  | 0.001 | 171.952   | 687.683   |
| C(workingday)[T.1] | 738.5017   | 301.509 | 2.449  | 0.014 | 147.554   | 1329.449  |
| temp               | 4629.9235  | 530.543 | 8.727  | 0.000 | 3590.078  | 5669.769  |
| hum                | -1477.5290 | 329.385 | -4.486 | 0.000 | -2123.112 | -831.946  |
| windspeed          | -3286.6508 | 501.515 | -6.553 | 0.000 | -4269.603 | -2303.699 |

|                |         |                   |          |
|----------------|---------|-------------------|----------|
| Omnibus:       | 113.944 | Durbin-Watson:    | 1.069    |
| Prob(Omnibus): | 0.000   | Jarque-Bera (JB): | 265.116  |
| Skew:          | -0.843  | Prob(JB):         | 2.70e-58 |
| Kurtosis:      | 5.421   | Cond. No.         | 37.7     |

Notes:

[1] Standard Errors are heteroscedasticity robust (HC3)

## ## 9.3 Regression Diagnostic Checks

```
import matplotlib.pyplot as plt
import numpy as np

fitted = model_revised.fittedvalues
residuals = model_revised.resid

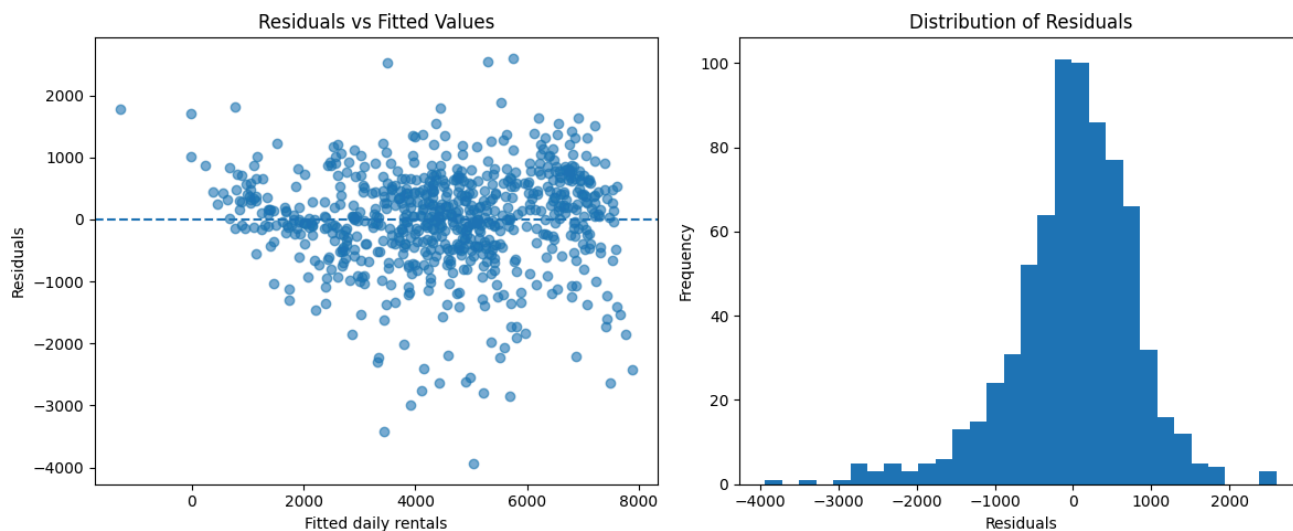
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Residuals vs fitted values
axes[0].scatter(fitted, residuals, alpha=0.6)
axes[0].axhline(0, linestyle="--")
axes[0].set_xlabel("Fitted daily rentals")
axes[0].set_ylabel("Residuals")
axes[0].set_title("Residuals vs Fitted Values")

# Histogram of residuals
axes[1].hist(residuals, bins=30)
axes[1].set_xlabel("Residuals")
axes[1].set_ylabel("Frequency")
axes[1].set_title("Distribution of Residuals")

plt.tight_layout()
plt.show()

print("Mean residual:", residuals.mean())
print("Residual SD:", residuals.std())
print("Durbin-Watson:", model_revised.durbin_watson)
```



Mean residual: 1.7870140084219655e-11

Residual SD: 801.0090780918331

```

AttributeError                                Traceback (most recent call last)
/tmp/ipykernel_46970/899328408.py in <cell line: 0>()
    27 print("Mean residual:", residuals.mean())
    28 print("Residual SD:", residuals.std())
--> 29 print("Durbin-Watson:", model_revised.durbin_watson)

/usr/local/lib/python3.13/dist-packages/statsmodels/base/wrapper.py in __getattr__(self, attr)
    33     pass
    34
--> 35     obj = getattr(results, attr)
    36     data = results.model.data
    37     how = self._wrap_attrs.get(attr)

```

AttributeError: 'OLSResults' object has no attribute 'durbin\_watson'

```

from statsmodels.stats.diagnostic import het_breuschpagan

bp_test = het_breuschpagan(
    residuals,
    model_revised.model.exog
)

bp_results = pd.Series(
    bp_test,
    index=[
        "LM statistic",
        "LM p-value",
        "F statistic",
        "F p-value"
    ]
)

bp_results

```

#### ## 9.4 Residual Autocorrelation Check

```

from statsmodels.stats.stattools import durbin_watson
from statsmodels.stats.diagnostic import acorr_ljungbox

dw = durbin_watson(residuals)

print("Durbin-Watson statistic:", round(dw, 3))

ljung_box = acorr_ljungbox(
    residuals,
    lags=[7, 14],

```

```

        return_df=True
    )

    ljung_box

```

#### ## 9.5 Distribution Check for Daily Rental Demand

```

print("Skewness of cnt:", round(df["cnt"].skew(), 3))
print("Kurtosis of cnt:", round(df["cnt"].kurt(), 3))

plt.figure(figsize=(8, 5))
plt.hist(df["cnt"], bins=30)
plt.xlabel("Daily bike rentals (cnt)")
plt.ylabel("Frequency")
plt.title("Distribution of Daily Bike Rentals")
plt.show()

```

#### ## 9.6 Main Weather Association Results

```

weather_results = pd.DataFrame({
    "Variable": ["temp", "hum", "windspeed"],
    "Coefficient": [
        model_revised.params["temp"],
        model_revised.params["hum"],
        model_revised.params["windspeed"]
    ],
    "Robust SE": [
        model_revised.bse["temp"],
        model_revised.bse["hum"],
        model_revised.bse["windspeed"]
    ],
    "p-value": [
        model_revised.pvalues["temp"],
        model_revised.pvalues["hum"],
        model_revised.pvalues["windspeed"]
    ],
    "CI lower": [
        model_revised.conf_int().loc["temp", 0],
        model_revised.conf_int().loc["hum", 0],
        model_revised.conf_int().loc["windspeed", 0]
    ],
    "CI upper": [
        model_revised.conf_int().loc["temp", 1],
        model_revised.conf_int().loc["hum", 1],
        model_revised.conf_int().loc["windspeed", 1]
    ]
})

weather_results.round(3)

```

```

print("R-squared:", round(model_revised.rsquared, 3))
print("Adjusted R-squared:", round(model_revised.rsquared_adj, 3))
print("Observations:", int(model_revised.nobs))
print("Durbin-Watson:", round(dw, 3))
print("Breusch-Pagan p-value:", f"{bp_results['LM p-value']:.3e}")

```

#### ## 10. Evidence and Claim Boundary

```

evidence_table = pd.DataFrame({
    "Evidence": [
        "No missing values",
        "No duplicate rows or dates",
        "No negative rental counts",
        "casual + registered = cnt for all observations",
        "Temperature has positive adjusted association with cnt",
        "Humidity has negative adjusted association with cnt",
        "Windspeed has negative adjusted association with cnt",
        "Residual heteroscedasticity detected",
        "Residual serial correlation detected"
    ],
    "Implication": [
        "No missing-value treatment was required",
        "Each daily record represents a unique observed day",
        "Rental-count values are internally valid on this check",
    ]
})

```

```

    "Target components are internally consistent",
    "Higher normalized temperature is associated with higher daily demand in the adjusted model",
    "Higher normalized humidity is associated with lower daily demand in the adjusted model",
    "Higher normalized windspeed is associated with lower daily demand in the adjusted model",
    "HC3 robust standard errors were used",
    "Model results should be interpreted as descriptive associations with temporal-dependence limitations"
  ]
})

evidence_table

```

## 10.1 Interpretation and Claim Boundary

The audit found no missing values, duplicate rows, repeated dates, negative rental counts, or inconsistencies between `casual + registered` and `cnt`. The adjusted association model explained 82.9% of the observed variation in daily rental demand ( $R^2 = 0.829$ ).

In the adjusted model, normalized temperature was positively associated with daily rental demand, while normalized humidity and windspeed were negatively associated with daily rental demand. These associations were statistically distinguishable from zero using HC3 robust standard errors.

The results should not be interpreted causally. The data are observational and represent daily system-level aggregates from 2011–2012. In addition, the diagnostic checks detected heteroscedasticity (Breusch–Pagan  $p < 0.001$ ) and residual serial correlation (Durbin–Watson = 1.069; Ljung–Box  $p < 0.001$  at both 7 and 14 lags). Therefore, the model provides evidence of association within the observed dataset but does not establish that weather conditions cause changes in rental demand.

The findings should also not be generalized automatically to other cities, bike-sharing systems, or time periods.

```

## 11.1 Software and Environment Versions

import sys
import platform
import pandas as pd
import numpy as np
import matplotlib
import statsmodels

environment = {
    "Python": sys.version.split()[0],
    "Platform": platform.platform(),
    "Pandas": pd.__version__,
    "NumPy": np.__version__,
    "Matplotlib": matplotlib.__version__,
    "Statsmodels": statsmodels.__version__
}

environment_df = pd.DataFrame(
    environment.items(),
    columns=["Software", "Version"]
)

environment_df

```

```

## 11.2 Final Frozen Dataset Verification

import hashlib
from pathlib import Path

data_path = Path("/content/day.csv")

file_bytes = data_path.read_bytes()
final_hash = hashlib.sha256(file_bytes).hexdigest()

print("File:", data_path.name)
print("Size (bytes):", len(file_bytes))
print("SHA-256:", final_hash)

expected_hash = "a6bcf826782d3c0fbfdcbecad17cd0884185a0dfe8ff10cd48a874ee7ba18be"

print("Hash matches frozen version:", final_hash == expected_hash)

```

11.3 Randomness and Reproducibility

The main audit and regression analysis use deterministic operations and do not rely on random sampling or randomized model fitting. Therefore, no random seed was required for the reported results.

The exact input data are frozen using the SHA-256 hash recorded in Section 2. The software environment is recorded above to support reproducibility.

12. Data Card

| Item                       | Description                                                                                                                                |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Dataset                    | UCI Bike Sharing Dataset — <code>day.csv</code>                                                                                            |
| Source                     | UCI Machine Learning Repository                                                                                                            |
| Dataset DOI                | 10.24432/C5W894                                                                                                                            |
| License                    | CC BY 4.0                                                                                                                                  |
| Study period               | 2011–2012                                                                                                                                  |
| Observations               | 731 daily records                                                                                                                          |
| Unit of analysis           | One calendar day                                                                                                                           |
| Target variable            | <code>cnt</code> — total daily bike rentals                                                                                                |
| Main variables of interest | <code>temp</code> , <code>hum</code> , <code>windspeed</code> , <code>weathersit</code>                                                    |
| Intended use               | Examine associations between observed weather conditions and daily bike-rental demand within the observed 2011–2012 Capital Bikeshare data |
| Out-of-scope use           | Causal claims about weather; automatic generalization to other cities, bike-sharing systems, or time periods                               |
| Known limitations          | Daily aggregate observations, observational design, unmeasured factors, normalized weather variables, and temporal dependence              |
| Leakage variables excluded | <code>casual</code> and <code>registered</code> , because they are components of <code>cnt</code>                                          |
| Frozen file                | <code>day.csv</code>                                                                                                                       |
| File size                  | 57,569 bytes                                                                                                                               |
| SHA-256                    | <code>a6bcf826782d3c0fbfdbcbeead17cd0884185a0dafe8ff10cd48a874ee7ba18be</code>                                                             |

12.1 Official Source Verification

The dataset was obtained from the UCI Machine Learning Repository. The official UCI record identifies the dataset as **Bike Sharing**, provides DOI **10.24432/C5W894**, and specifies a **Creative Commons Attribution 4.0 International (CC BY 4.0)** license. The repository describes the dataset as containing daily and hourly rental-bike counts from the Capital Bikeshare system for 2011–2012 with corresponding weather and seasonal information.

The analysis uses the downloaded `day.csv` file rather than modifying or reconstructing the dataset. The exact file used in the analysis is identified by its SHA-256 hash recorded in the frozen-version section.

13. Data Dictionary

The following definitions were verified against the dataset's `Readme.txt`. The daily file (`day.csv`) does not contain the hourly variable `hr`.

| Variable                |
|-------------------------|
| <code>instant</code>    |
| <code>dteday</code>     |
| <code>season</code>     |
| <code>yr</code>         |
| <code>mnth</code>       |
| <code>holiday</code>    |
| <code>weekday</code>    |
| <code>workingday</code> |
| <code>weathersit</code> |
| <code>temp</code>       |
| <code>atemp</code>      |
| <code>hum</code>        |
| <code>windspeed</code>  |
| <code>casual</code>     |
| <code>registered</code> |
| <code>cnt</code>        |

**Note:** The README defines `weathersit` category 4, but the frozen `day.csv` used in this audit contains only categories 1, 2, and 3. The range audit confirmed that no category-4 observations were present.

## 14. AI-Use and Independent-Verification Log

### AI-use log

**AI tool used:** ChatGPT

#### Tasks for which AI assistance was used:

- Refining the research question and identifying the stakeholder, population, sample, unit, target, and estimand.
- Suggesting an appropriate audit structure for provenance, schema, range, missingness, duplicate, anomaly, and leakage checks.
- Suggesting Python code for data auditing, exploratory analysis, regression modeling, and diagnostics.
- Helping interpret statistical outputs and identify limitations of the observational design.
- Helping organize the Data Card, Data Dictionary, reproducibility section, and claim boundaries.

#### Accepted AI assistance:

- Suggestions for audit structure and reproducible Python implementation were used.
- Suggestions to distinguish association from causation were incorporated.
- Suggestions to exclude `casual` and `registered` from predictors because they are components of `cnt` were incorporated.
- Suggestions to use HC3 robust standard errors after detecting heteroscedasticity were incorporated.

#### Rejected or modified AI advice:

- The initial regression specification was not retained after the executed model showed rank deficiency.
- Interpretations were restricted to the actual outputs produced in the notebook.
- No AI-generated numerical result was accepted without executing the corresponding code on the frozen dataset.

### Independent verification

The analysis was independently executed in the notebook using the frozen `day.csv` file. The reported dataset size and SHA-256 hash were checked against the frozen version, and the hash matched.

The audit outputs were reviewed directly after execution, including schema, range, missingness, duplicate, anomaly, leakage, regression, and diagnostic results. The final interpretations were based on these executed outputs rather than on AI-generated values.

The notebook should be rerun from top to bottom before submission to confirm that the reported results remain reproducible.

## 15. Failed Runs and Negative Results

### 15.1 Initial regression specification

The initial regression model included `holiday`, `weekday`, and `workingday` together with the other predictors. The executed model produced a rank-deficiency warning and an extremely large condition number, indicating a singular or nearly singular design matrix.

The model was therefore not used as the final specification. The calendar-variable specification was revised by removing `holiday`, after which the revised model no longer produced the rank-deficiency warning and had a condition number of 37.7.

This failed specification was retained as part of the audit record rather than removed from the notebook.

### 15.2 Failed diagnostic attempt

An initial attempt to obtain the Durbin-Watson statistic using an attribute that was not available in the `OLSResults` object produced an `AttributeError`.

The statistic was subsequently calculated using an appropriate diagnostic function, giving a Durbin-Watson statistic of 1.069. The failed attempt was retained as part of the reproducibility record.

### 15.3 Negative audit results

Several checks produced negative findings that are important to the interpretation:

- No missing values were detected.
- No completely duplicated rows were detected.
- No repeated dates were detected.
- No negative rental counts were detected.

- `casual + registered` equaled `cnt` for all 731 observations.
- No unexpected categorical values were detected.
- IQR screening produced potential outlier flags for some variables, but these were not automatically treated as data errors.
- Residual diagnostics detected heteroscedasticity and serial correlation.

These negative and diagnostic results were retained because they define the evidence boundaries of the analysis.

## ✓ 16. Final Audit Conclusion

The frozen UCI Bike Sharing `day.csv` dataset contains 731 daily observations and 16 variables, with no missing values, duplicate rows, repeated dates, unexpected categorical values, or negative rental counts detected in the audit. The relationship `casual + registered = cnt` was also satisfied for every observation.

The analysis supports an associational question about daily bike-rental demand and observed weather conditions. In the adjusted model, normalized temperature was positively associated with daily rental demand, whereas normalized humidity and windspeed were negatively associated with demand. The model had an  $R^2$  of 0.829.

However, the evidence has important boundaries. The data are observational daily aggregates from 2011–2012, so the results do not establish causal effects. The diagnostic analysis also detected heteroscedasticity and residual serial correlation. Therefore, the findings describe associations within the observed dataset and should not automatically be generalized to other cities, bike-sharing systems, or time periods.

```
from pathlib import Path

output_dir = Path("/content/outputs")
output_dir.mkdir(exist_ok=True)

print("Output directory:", output_dir)
```

```
# Check which audit variables currently exist

variables_to_check = [
    "output_dir",
    "schema_df",
    "range_df",
    "missingness_df",
    "anomaly_summary",
    "analysis_df"
]

for var in variables_to_check:
    print(f"{var}: {'FOUND' if var in globals() else 'NOT FOUND'}")
```

```
# Recreate schema audit table from the frozen dataset

schema_df = pd.DataFrame({
    "column": df.columns,
    "dtype": [str(df[col].dtype) for col in df.columns],
    "non_null": [df[col].notna().sum() for col in df.columns],
    "unique_values": [df[col].nunique() for col in df.columns]
})

display(schema_df)
```

```
# Recreate numeric range audit table

numeric_cols = df.select_dtypes(include="number").columns

range_df = pd.DataFrame({
    "column": numeric_cols,
    "min": [df[col].min() for col in numeric_cols],
    "max": [df[col].max() for col in numeric_cols]
})

display(range_df)
```

```
print("schema_df:", "FOUND" if "schema_df" in globals() else "NOT FOUND")
print("range_df:", "FOUND" if "range_df" in globals() else "NOT FOUND")
print("anomaly_summary:", "FOUND" if "anomaly_summary" in globals() else "NOT FOUND")
```

```
print("analysis_df:", "FOUND" if "analysis_df" in globals() else "NOT FOUND")
```

```
# Save raw audit results for reproducibility

schema_df.to_csv(
    output_dir / "schema_audit.csv",
    index=False
)

range_df.to_csv(
    output_dir / "range_audit.csv",
    index=False
)

missingness_df.to_csv(
    output_dir / "missingness_audit.csv",
    index=False
)

anomaly_summary.to_csv(
    output_dir / "anomaly_audit.csv",
    index=False
)

analysis_df.to_csv(
    output_dir / "analysis_dataset.csv",
    index=False
)

print("Saved audit result files:")
for file in sorted(output_dir.iterdir()):
    print("-", file.name)
```

```
# Recreate missingness audit table

missingness_df = pd.DataFrame({
    "column": df.columns,
    "missing_count": [df[col].isna().sum() for col in df.columns],
    "missing_percent": [
        df[col].isna().mean() * 100 for col in df.columns
    ]
})

display(missingness_df)
```

```
print(
    "missingness_df:",
    "FOUND" if "missingness_df" in globals() else "NOT FOUND"
)
```

```
# Save raw audit results for reproducibility

schema_df.to_csv(
    output_dir / "schema_audit.csv",
    index=False
)

range_df.to_csv(
    output_dir / "range_audit.csv",
    index=False
)

missingness_df.to_csv(
    output_dir / "missingness_audit.csv",
    index=False
)

anomaly_summary.to_csv(
    output_dir / "anomaly_audit.csv",
    index=False
)

analysis_df.to_csv(
    output_dir / "analysis_dataset.csv",
```

```

        index=False
    )

    print("Saved audit result files:")
    for file in sorted(output_dir.iterdir()):
        print("-", file.name)

```

```

# Check that the final revised model is available

print(
    "model_revised:",
    "FOUND" if "model_revised" in globals() else "NOT FOUND"
)

```

```

# Save final regression results and model summary

# 1. Coefficients, robust SEs, p-values, and confidence intervals
coef_table = pd.DataFrame({
    "coefficient": model_revised.params,
    "robust_se": model_revised.bse,
    "p_value": model_revised.pvalues,
    "ci_lower": model_revised.conf_int()[0],
    "ci_upper": model_revised.conf_int()[1]
})

coef_table.to_csv(
    output_dir / "model_coefficients.csv"
)

# 2. Full model summary
with open(output_dir / "model_summary.txt", "w") as f:
    f.write(model_revised.summary().as_text())

print("Saved model result files:")
print("- model_coefficients.csv")
print("- model_summary.txt")

```

```

# Save model diagnostic results

from statsmodels.stats.diagnostic import het_breuschpagan, acorr_ljungbox
from statsmodels.stats.stattools import durbin_watson

# Residuals
residuals = model_revised.resid

# Breusch-Pagan test
bp_test = het_breuschpagan(
    residuals,
    model_revised.model.exog
)

bp_results = pd.DataFrame({
    "test": [
        "Breusch-Pagan LM statistic",
        "Breusch-Pagan LM p-value",
        "Breusch-Pagan F statistic",
        "Breusch-Pagan F p-value"
    ],
    "value": bp_test
})

bp_results.to_csv(
    output_dir / "breusch_pagan_test.csv",
    index=False
)

# Durbin-Watson
dw_stat = durbin_watson(residuals)

dw_results = pd.DataFrame({
    "test": ["Durbin-Watson"],
    "value": [dw_stat]
})

dw_results.to_csv(

```

```

        output_dir / "durbin_watson_test.csv",
        index=False
    )

    # Ljung-Box tests
    ljungbox_results = acorr_ljungbox(
        residuals,
        lags=[7, 14],
        return_df=True
    )

    ljungbox_results.to_csv(
        output_dir / "ljung_box_test.csv"
    )

    print("Saved diagnostic files:")
    print("- breusch_pagan_test.csv")
    print("- durbin_watson_test.csv")
    print("- ljung_box_test.csv")

```

```

# Save reproducibility and environment information

import sys
import platform
import hashlib
import importlib.metadata as md
from pathlib import Path

data_path = Path("/content/day.csv")

# Calculate SHA-256 of the frozen dataset
sha256_hash = hashlib.sha256(data_path.read_bytes()).hexdigest()

# Package versions used in the analysis
packages = [
    "pandas",
    "numpy",
    "matplotlib",
    "statsmodels"
]

environment_lines = [
    f"Python: {sys.version}",
    f"Platform: {platform.platform()}",
    "",
    "Package versions:"
]

for package in packages:
    environment_lines.append(
        f"{package}: {md.version(package)}"
    )

environment_lines.extend([
    "",
    "Frozen dataset:",
    f"File: {data_path.name}",
    f"Size (bytes): {data_path.stat().st_size}",
    f"SHA-256: {sha256_hash}",
    f"Expected SHA-256: a6bcf826782d3c0fbfdcbecad17cd0884185a0dfe8ff10cd48a874ee7ba18be",
    f"Hash matches frozen version: {sha256_hash == 'a6bcf826782d3c0fbfdcbecad17cd0884185a0dfe8ff10cd48a874ee7ba18be'}"
])

environment_text = "\n".join(environment_lines)

(output_dir / "environment_and_hash.txt").write_text(
    environment_text,
    encoding="utf-8"
)

print(environment_text)

```

## ✓ AI Use and Independent Verification Log

## AI tool used

ChatGPT was used as a methodological and writing assistant during the audit.

## Tasks supported by AI

- Clarifying the assignment requirements and audit workflow.
- Helping define the research question, stakeholder, population, sample, unit of analysis, target, and estimand.
- Suggesting data-quality checks for schema, ranges, missingness, duplicates, anomalies, and target leakage.
- Helping interpret statistical diagnostics and identify appropriate claim boundaries.
- Helping organize reproducibility outputs and documentation.
- Reviewing wording for clarity and avoiding causal claims from observational data.

## AI advice accepted

The suggested audit structure, leakage check, diagnostic checks, reproducibility documentation, and cautious interpretation of associations were used where they matched the assignment requirements and the observed notebook results.

## AI advice rejected or corrected

AI-generated variable names and code were not assumed to exist in the notebook. When a suggested variable such as `schema_df`, `range_df`, or `missingness_df` was not available, the audit table was recreated from the actual frozen dataset rather than treating an assumed result as valid.

An initially fitted model showed rank deficiency. It was not treated as the final model; the revised specification was used after checking the actual model output.

## Independent verification

All reported numerical results were independently verified by executing the Python code in the notebook and checking the resulting tables, model output, diagnostic tests, and dataset hash. The frozen `day.csv` file was verified using SHA-256, which matched the recorded expected hash.

AI was not used as a substitute for execution of the analysis or verification of numerical results.

```
from pathlib import Path

print("Current output files:")
for file in sorted(Path("/content/outputs").iterdir()):
    print("-", file.name)

print("\nTotal files:", len(list(Path("/content/outputs").iterdir())))
```

```
# Save the main software dependencies used in the analysis

requirements_text = """pandas==2.2.3
numpy==2.1.3
matplotlib==3.10.0
statsmodels==0.15.0
"""

(output_dir / "requirements.txt").write_text(
    requirements_text,
    encoding="utf-8"
)

print(requirements_text)
print("Saved:", output_dir / "requirements.txt")
```

```
print("requirements.txt exists:",
      (output_dir / "requirements.txt").exists())
```

```
# Create README.md for the project repository

readme_text = """# DS-A1 | Data Provenance and Measurement Audit

## Dataset

This audit uses the UCI Bike Sharing Dataset, specifically the frozen `day.csv` file containing daily observations from the Cap
```

Source: UCI Machine Learning Repository  
Dataset DOI: 10.24432/C5W894  
License: CC BY 4.0

## ## Research Question

How is daily bike-rental demand associated with observed weather conditions in the UCI Bike Sharing Dataset?

## ## Study Design

- Stakeholder: Bike-sharing system operator
- Population: Daily bike-sharing activity represented by Capital Bikeshare during 2011-2012
- Sample: 731 daily observations
- Unit of analysis: One calendar day
- Target: `cnt`, total daily bike rentals
- Estimand: Association between observed daily weather conditions and total daily bike-rental demand within the observed 2011-2012

## ## Data Provenance and Freeze

File: `day.csv`  
Size: 57,569 bytes  
SHA-256: `a6bcf826782d3c0fbfdcbcead17cd0884185a0daffe8ff10cd48a874ee7ba18be`

The hash was independently recomputed in the notebook and matched the recorded frozen version.

## ## Audit Scope

The audit includes:

- Schema and data-type checks
- Range and validity checks
- Missingness checks
- Duplicate-row and duplicate-date checks
- IQR-based anomaly screening
- Logical consistency checks
- Target-leakage checks
- Predictor correlation and redundancy checks
- Regression diagnostics
- Reproducibility and environment checks
- Bias and measurement limitations
- Claim boundaries

## ## Main Analysis

The adjusted model examines the association between daily rental demand and:

- Temperature
- Humidity
- Windspeed
- Weather situation
- Year
- Month
- Weekday
- Working-day status

The variables `casual` and `registered` were excluded as predictors because they are direct components of the target `cnt` and

## ## Main Result Boundary

The adjusted model explains 82.9% of the observed variation in daily rental demand ( $R^2 = 0.829$ ). Within the observed dataset, r

These are associations, not causal effects. The data are observational daily aggregates from 2011-2012. Diagnostic checks also

The findings should not automatically be generalized to other cities, bike-sharing systems, or time periods.

## ## Reproducibility

Main software environment:

- Python 3.13.15
- pandas 2.2.3
- NumPy 2.1.3
- Matplotlib 3.10.0
- statsmodels 0.15.0

The required package versions are recorded in `requirements.txt`.

## ## Raw Evidence

The `outputs/` directory contains the audit tables, model results, diagnostic results, environment information, and frozen-data

## ## AI Use

ChatGPT was used as a methodological and writing assistant. Numerical results were verified by executing the analysis code in t  
"""

```
(output_dir / "README.md").write_text(
    readme_text,
    encoding="utf-8"
)
```

```
print("README created:", output_dir / "README.md")
```

```
print((output_dir / "README.md").read_text(encoding="utf-8"))
```

## # Create Data Card

```
data_card_text = """# Data Card
```

## ## Dataset

```
UCI Bike Sharing Dataset - `day.csv`
```

## ## Source

```
UCI Machine Learning Repository
```

```
Dataset DOI: 10.24432/C5W894
```

```
License: CC BY 4.0
```

## ## Data Period

```
2011-2012
```

## ## Sample

```
731 daily observations
```

## ## Unit of Analysis

```
One calendar day
```

## ## Stakeholder

```
Bike-sharing system operator
```

## ## Target

```
`cnt` - total number of daily bike rentals, including casual and registered users.
```

## ## Main Variables

```
The main analysis uses weather and calendar variables including:
```

- `temp` - normalized temperature
- `hum` - normalized humidity
- `windspeed` - normalized wind speed
- `weathersit` - weather situation category
- `yr` - year indicator
- `mnth` - month
- `weekday` - day of week
- `workingday` - working-day indicator

## ## Intended Use

```
To examine associations between observed weather conditions and daily bike-rental demand within the observed Capital Bikeshare
```

## ## Out-of-Scope Uses

```
The dataset should not be used in this audit to:
```

- establish causal effects of weather on bike-rental demand;
- claim that weather is the sole explanation for changes in demand;
- automatically generalize the results to other cities or bike-sharing systems;
- automatically generalize the results to other time periods.

## ## Known Limitations

- The observations are daily system-level aggregates.
- Individual users' reasons for renting or not renting are not recorded.
- Other factors affecting demand may not be represented.

- Weather variables are normalized measurements.
- The data cover 2011-2012 and may not represent current bike-sharing behavior.
- Temporal dependence is present in the observations.
- Diagnostic checks detected heteroscedasticity and residual serial correlation.

## ## Leakage Variables

`casual` and `registered` were excluded from the regression predictors because:

`casual + registered = cnt`

for every observation. Including these variables as predictors would directly expose components of the target.

## ## Frozen Dataset

File: `day.csv`

Size: 57,569 bytes

SHA-256:

`a6bcf826782d3c0fbfdcbceead17cd0884185a0dafa8ff10cd48a874ee7ba18be`

The hash was independently recomputed and matched the recorded frozen version.

"""

```
(output_dir / "data_card.md").write_text(
    data_card_text,
    encoding="utf-8"
)
```

```
print("Data Card created:", output_dir / "data_card.md")
```

```
print((output_dir / "data_card.md").read_text(encoding="utf-8"))
```

## # Create Data Dictionary

data\_dictionary\_text = """# Data Dictionary

| Variable     | Type        | Description                                                         | Role / Notes                                                        |
|--------------|-------------|---------------------------------------------------------------------|---------------------------------------------------------------------|
| `instant`    | Integer     | Record index                                                        | Identifier; not used as an analysis predictor                       |
| `dteday`     | Date/string | Calendar date                                                       | Date identifier; one unique date per observation                    |
| `season`     | Integer     | Season category coded 1-4 according to the dataset documentation    | Calendar variable                                                   |
| `yr`         | Integer     | Year indicator: 0 = 2011, 1 = 2012                                  | Calendar variable                                                   |
| `mnth`       | Integer     | Month, coded 1-12                                                   | Calendar variable                                                   |
| `holiday`    | Integer     | Holiday indicator                                                   | Calendar variable                                                   |
| `weekday`    | Integer     | Day-of-week category, coded 0-6                                     | Calendar variable                                                   |
| `workingday` | Integer     | 1 if neither weekend nor holiday; otherwise 0                       | Calendar variable                                                   |
| `weathersit` | Integer     | Weather situation category                                          | Weather variable; frozen daily data contain observed categories 1-3 |
| `temp`       | Float       | Normalized temperature measurement                                  | Weather predictor                                                   |
| `atemp`      | Float       | Normalized feeling-temperature measurement                          | Weather variable; highly correlated with `temp`                     |
| `hum`        | Float       | Normalized humidity measurement                                     | Weather predictor                                                   |
| `windspeed`  | Float       | Normalized wind-speed measurement                                   | Weather predictor                                                   |
| `casual`     | Integer     | Count of casual users                                               | Component of `cnt`; excluded as predictor to prevent target leakage |
| `registered` | Integer     | Count of registered users                                           | Component of `cnt`; excluded as predictor to prevent target leakage |
| `cnt`        | Integer     | Total number of rental bikes, including casual and registered users | Target variable                                                     |

## ## Notes

The frozen `day.csv` file contains 731 daily observations and 16 variables.

The README for the dataset defines additional weather-category information for `weathersit`; however, the frozen daily file use

The hourly variable `hr` is not present in `day.csv` and was therefore not included in this analysis.